ICT203 ASSIGNMENT 2

ICT203 Artificial Intelligence and Intelligent Agents

Name: Rohit Kumar Malik
Student ID: 35363998

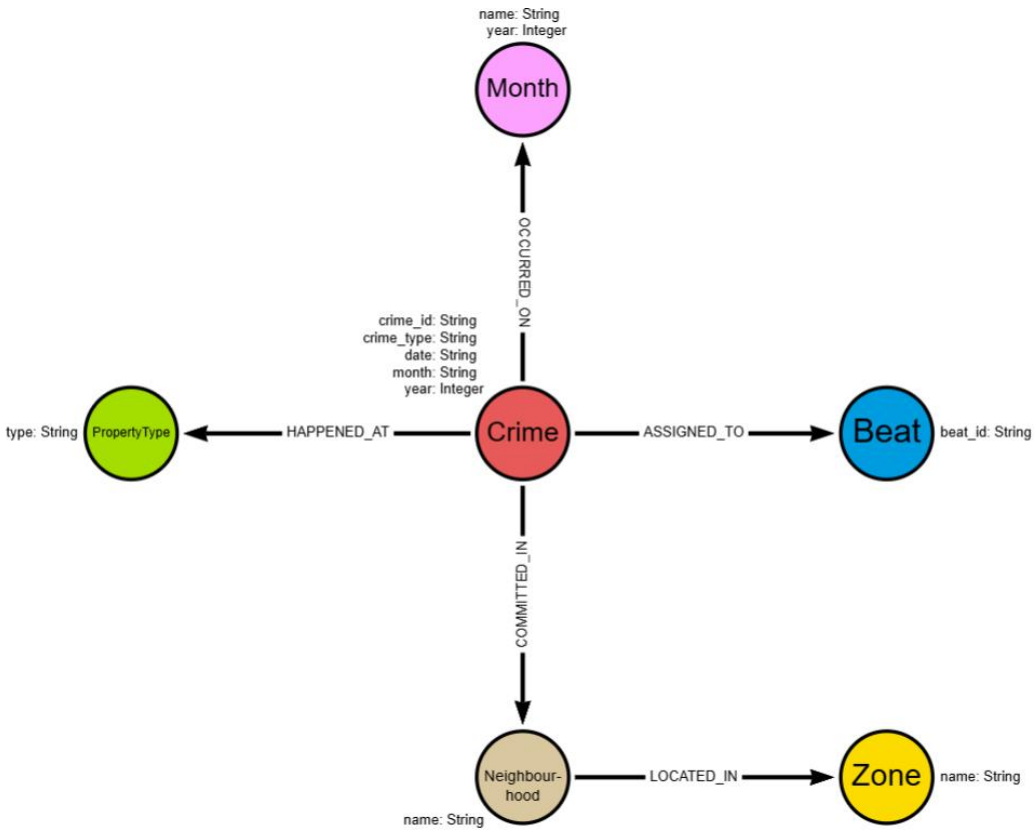
Name: Rohit Kumar Malik
Student ID: 35363998

Table of Contents

Property Graph Schema (Arrow Tool) 2
Python Preprocessing Script 3
Python Code Screenshot 3
Terminal Output Screenshot 4
Generated CSV Files (Node & Relationship) 4
Database Creation with Cypher 5
Creating constraints for uniqueness 5
Importing Nodes 5
a) Importing Crime Nodes 5
b) Importing Neighbourhood Nodes 5
c) Importing Zone Nodes 5
d) Importing PropertyType Nodes 5
e) Importing Beat Nodes 6
f) Importing Month Nodes 6
Creating Relationships 6
a) COMMITTED_IN (Crime → Neighbourhood) 6
b) LOCATED_IN (Neighbourhood → Zone) 6
c) HAPPENED_AT (Crime → PropertyType) 6
d) OCCURRED_ON (Crime → Month) 6
e) ASSIGNED_TO (Crime → Beat) 7
Verifying Node and Relationship Counts 7
Verification of different colors for each node label 7
Graph Structure Visualisation 8
Provided Cypher Queries & Visualisations 9
Count Crimes by Type, Neighbourhood, and Month 9
Neighbourhoods Sharing Same Crime Types 9
Top 5 Neighbourhoods for a Specific Crime 11
Crime Types per Property Type 12
Highest Crime Month per Beat 13
Zones with Shared High-Crime Months 14
Custom Queries with Pathfinding 15
Shortest Path Between High-Crime Neighbourhoods 15
Pattern Between Different Crime Types Through Locations 16
Design Choices – Discussion and Justification 17

Name: Rohit Kumar Malik
Student ID: 35363998

Property Graph Schema (Arrow Tool)



Name: Rohit Kumar Malik
Student ID: 35363998

Python Preprocessing Script

Python Code Screenshot

Below is the screenshot of the Python script. It reads the data from crime_subset.csv file, process the data, and then generates all the node and relationship CSV files. I've also added comments and used proper variable names for easy understanding.

```python
import pandas as pd
import numpy as np
from datetime import datetime
import os

# Function to create directory if it doesn't exist
def ensure_directory(directory):
    """
    Create directory if it doesn't exist

    Args:
        directory (str): Path to directory
    """
    if not os.path.exists(directory):
        os.makedirs(directory)

# Set the output directory for CSV files
output_dir = "csv_output"
ensure_directory(output_dir)

# Load the dataset (Only using first 100 rows as provided, but code works for full dataset)
print("Loading dataset...")
df = pd.read_csv('crime_subset.csv')  # Replace with your actual filename
df.rename(columns={
    'type': 'crime_type',
    'neighborhood': 'neighbourhood',
    'npu': 'zone',
    'location': 'property_type',
    'beat': 'beat'
}, inplace=True)
print(f"Loaded {len(df)} records")

# Data cleaning and preprocessing
print("Cleaning and preprocessing data...")

# Convert date strings to datetime objects and extract month and year
df['date'] = pd.to_datetime(df['date'], errors='coerce')
df['month'] = df['date'].dt.month
df['year'] = df['date'].dt.year
df['month_name'] = df['date'].dt.strftime('%B')  # Full month name

# Create unique IDs where needed
df['crime_id'] = df.index  # Use index as crime_id if not present in original data
```

```python

# Generate CSV for Crime nodes
print("Generating Crime nodes CSV...")
crime_nodes = df[['crime_id', 'crime_type', 'date', 'month', 'year']].drop_duplicates()
crime_nodes['date'] = crime_nodes['date'].dt.strftime('%Y-%m-%d')  # Format date as string
crime_nodes.to_csv(f"{output_dir}/crime_nodes.csv", index=False)

# Generate CSV for Neighbourhood nodes
print("Generating Neighbourhood nodes CSV...")
neighbourhood_nodes = df[['neighbourhood']].drop_duplicates()
neighbourhood_nodes.rename(columns={'neighbourhood': 'name'}, inplace=True)
neighbourhood_nodes.to_csv(f"{output_dir}/neighbourhood_nodes.csv", index=False)

# Generate CSV for Zone nodes
print("Generating Zone nodes CSV...")
zone_nodes = df[['zone']].drop_duplicates()
zone_nodes.rename(columns={'zone': 'name'}, inplace=True)
zone_nodes.to_csv(f"{output_dir}/zone_nodes.csv", index=False)

# Generate CSV for PropertyType nodes
print("Generating PropertyType nodes CSV...")
property_nodes = df[['property_type']].drop_duplicates()
property_nodes.rename(columns={'property_type': 'type'}, inplace=True)
property_nodes.to_csv(f"{output_dir}/property_type_nodes.csv", index=False)

# Generate CSV for Beat nodes
print("Generating Beat nodes CSV...")
beat_nodes = df[['beat']].drop_duplicates()
beat_nodes.rename(columns={'beat': 'beat_id'}, inplace=True)
beat_nodes.to_csv(f"{output_dir}/beat_nodes.csv", index=False)

# Generate CSV for Month nodes
print("Generating Month nodes CSV...")
month_nodes = df[['month_name', 'year']].drop_duplicates()
month_nodes.to_csv(f"{output_dir}/month_nodes.csv", index=False)

# Generate relationship CSVs
print("Generating relationship CSVs...")

# COMMITTED_IN relationship (Crime → Neighbourhood)
crime_neighbourhood_rel = df[['crime_id', 'neighbourhood']].rename(
    columns={'neighbourhood': 'neighbourhood_name'}
)
crime_neighbourhood_rel.to_csv(f"{output_dir}/committed_in_relationships.csv", index=False)

```

Name: Rohit Kumar Malik
Student ID: 35363998

```python

# LOCATED_IN relationship (Neighbourhood → Zone)
neighbourhood_zone_rel = df[['neighbourhood', 'zone']].drop_duplicates().rename(
    columns={'neighbourhood': 'neighbourhood_name', 'zone': 'zone_name'}
)
neighbourhood_zone_rel.to_csv(f"{output_dir}/located_in_relationships.csv", index=False)

# HAPPENED_AT relationship (Crime → PropertyType)
crime_property_rel = df[['crime_id', 'property_type']].rename(
    columns={'property_type': 'property_type_type'}
)
crime_property_rel.to_csv(f"{output_dir}/happened_at_relationships.csv", index=False)

# OCCURRED_ON relationship (Crime → Month)
crime_month_rel = df[['crime_id', 'month_name', 'year']]
crime_month_rel.to_csv(f"{output_dir}/occurred_on_relationships.csv", index=False)

# ASSIGNED_TO relationship (Crime → Beat)
crime_beat_rel = df[['crime_id', 'beat']].rename(
    columns={'beat': 'beat_id'}
)
crime_beat_rel.to_csv(f"{output_dir}/assigned_to_relationships.csv", index=False)

print("CSV generation complete. Files are saved in the 'csv_output' directory.")
print(f"Generated {len(os.listdir(output_dir))} CSV files.")
```

Terminal Output Screenshot

Below is the screenshot of the output from the terminal after I ran the script. It shows that the code successfully loaded and the required 11 CSV files are generated, without any errors.

```
EXPLORER
QUESTION 2
  csv_output
  crime_subset.csv
  q2.py
q2.py > ...
import pandas as pd
import numpy as np
from datetime import datetime
import os

# Function to create directory if it doesn't exist
def ensure_directory(directory):
    """
    Create directory if it doesn't exist

    Args:
        directory (str): Path to directory
    """
    if not os.path.exists(directory):
        os.makedirs(directory)

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS D:\ICT 203\Assignmnet\Question 2> conda activate ict203
PS D:\ICT 203\Assignmnet\Question 2> python q2.py
Loading dataset...
Loaded 24386 records
Cleaning and preprocessing data...
D:\ICT 203\Assignmnet\Question 2\q2.py:37: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, pleas
e specify a format.
  df['date'] = pd.to_datetime(df['date'], errors='coerce')
Generating Crime nodes CSV...
Generating Neighbourhood nodes CSV...
Generating Zone nodes CSV...
Generating PropertyType nodes CSV...
Generating Beat nodes CSV...
Generating Month nodes CSV...
Generating relationship CSVs...
CSV generation complete. Files are saved in the 'csv_output' directory.
Generated 11 CSV files.
PS D:\ICT 203\Assignmnet\Question 2>
```

Generated CSV Files (Node & Relationship)

This is the folder which was generated after running the script, all the CSV files were saved. Each file matches a specific node or relationship type based on the schema I created earlier. In total, there are 11 CSVs, which include 5 node CSVs files and the 6 relationships between them.

assigned_to_relationships.csv	4/6/2025 1:07 PM	Microsoft Excel Co...	201 KB
beat_nodes.csv	4/6/2025 1:07 PM	Microsoft Excel Co...	1 KB
committed_in_relationships.csv	4/6/2025 1:07 PM	Microsoft Excel Co...	263 KB
crime_nodes.csv	4/6/2025 1:07 PM	Microsoft Excel Co...	459 KB
happened_at_relationships.csv	4/6/2025 1:07 PM	Microsoft Excel Co...	322 KB
located_in_relationships.csv	4/6/2025 1:07 PM	Microsoft Excel Co...	5 KB
month_nodes.csv	4/6/2025 1:07 PM	Microsoft Excel Co...	1 KB
neighbourhood_nodes.csv	4/6/2025 1:07 PM	Microsoft Excel Co...	4 KB
occurred_on_relationships.csv	4/6/2025 1:07 PM	Microsoft Excel Co...	296 KB
property_type_nodes.csv	4/6/2025 1:07 PM	Microsoft Excel Co...	115 KB
zone_nodes.csv	4/6/2025 1:07 PM	Microsoft Excel Co...	1 KB

Name: Rohit Kumar Malik
Student ID: 35363998

Database Creation with Cypher

Creating constraints for uniqueness

```
CREATE CONSTRAINT crime_id_constraint IF NOT EXISTS FOR (c:Crime) REQUIRE c.crime_id IS UNIQUE;
CREATE CONSTRAINT neighbourhood_name_constraint IF NOT EXISTS FOR (n:Neighbourhood) REQUIRE n.name IS UNIQUE;
CREATE CONSTRAINT zone_name_constraint IF NOT EXISTS FOR (z:Zone) REQUIRE z.name IS UNIQUE;
CREATE CONSTRAINT property_type_constraint IF NOT EXISTS FOR (p:PropertyType) REQUIRE p.type IS UNIQUE;
CREATE CONSTRAINT beat_id_constraint IF NOT EXISTS FOR (b:Beat) REQUIRE b.beat_id IS UNIQUE;
CREATE CONSTRAINT month_year_constraint IF NOT EXISTS FOR (m:Month) REQUIRE (m.month_name, m.year) IS UNIQUE;
```

Importing Nodes

a) Importing Crime Nodes

```
LOAD CSV WITH HEADERS FROM 'file:///crime_nodes.csv' AS row
CREATE (c:Crime {
    crime_id: toInteger(row.crime_id),
    crime_type: row.crime_type,
    date: date(row.date),
    month: toInteger(row.month),
    year: toInteger(row.year)
})
SET c:Crime
RETURN count(c);
```

b) Importing Neighbourhood Nodes

```
LOAD CSV WITH HEADERS FROM 'file:///neighbourhood_nodes.csv' AS row
CREATE (n:Neighbourhood {
    name: row.name
})
SET n:Neighbourhood
RETURN count(n);
```

c) Importing Zone Nodes

```
LOAD CSV WITH HEADERS FROM 'file:///zone_nodes.csv' AS row
CREATE (z:Zone {
    name: row.name
})
SET z:Zone
RETURN count(z);
```

d) Importing PropertyType Nodes

```
LOAD CSV WITH HEADERS FROM 'file:///property_type_nodes.csv' AS row
CREATE (p:PropertyType {
    type: row.type
})
SET p:PropertyType
RETURN count(p);
```

Name: Rohit Kumar Malik
Student ID: 35363998

e) Importing Beat Nodes

```
LOAD CSV WITH HEADERS FROM 'file:///beat_nodes.csv' AS row
CREATE (b:Beat {
    beat_id: row.beat_id
})
SET b:Beat
RETURN count(b);
```

f) Importing Month Nodes

```
LOAD CSV WITH HEADERS FROM 'file:///month_nodes.csv' AS row
CREATE (m:Month {
    month_name: row.month_name,
    year: toInteger(row.year)
})
SET m:Month
RETURN count(m);
```

Creating Relationships

a) COMMITTED_IN (Crime → Neighbourhood)

```
LOAD CSV WITH HEADERS FROM 'file:///committed_in_relationships.csv' AS row
MATCH (c:Crime {crime_id: toInteger(row.crime_id)})
MATCH (n:Neighbourhood {name: row.neighbourhood_name})
CREATE (c)-[:COMMITTED_IN]->(n)
RETURN count(*);
```

b) LOCATED_IN (Neighbourhood → Zone)

```
LOAD CSV WITH HEADERS FROM 'file:///located_in_relationships.csv' AS row
MATCH (n:Neighbourhood {name: row.neighbourhood_name})
MATCH (z:Zone {name: row.zone_name})
CREATE (n)-[:LOCATED_IN]->(z)
RETURN count(*);
```

c) HAPPENED_AT (Crime → PropertyType)

```
LOAD CSV WITH HEADERS FROM 'file:///happened_at_relationships.csv' AS row
MATCH (c:Crime {crime_id: toInteger(row.crime_id)})
MATCH (p:PropertyType {type: row.property_type_type})
CREATE (c)-[:HAPPENED_AT]->(p)
RETURN count(*);
```

d) OCCURRED_ON (Crime → Month)

```
LOAD CSV WITH HEADERS FROM 'file:///occurred_on_relationships.csv' AS row
MATCH (c:Crime {crime_id: toInteger(row.crime_id)})
MATCH (m:Month {month_name: row.month_name, year: toInteger(row.year)})
CREATE (c)-[:OCCURRED_ON]->(m)
RETURN count(*);
```

Name: Rohit Kumar Malik
Student ID: 35363998

e) ASSIGNED_TO (Crime → Beat)

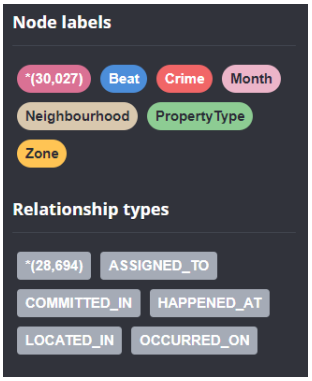
```
LOAD CSV WITH HEADERS FROM 'file:///assigned_to_relationships.csv' AS row
MATCH (c:Crime {crime_id: toInteger(row.crime_id)})
MATCH (b:Beat {beat_id: row.beat_id})
CREATE (c)-[:ASSIGNED_TO]->(b)
RETURN count(*);
```

Verifying Node and Relationship Counts

```
MATCH (n)
RETURN labels(n) as NodeType, count(*) as Count
ORDER BY Count DESC;
```

	NodeType	Count
1	["Crime"]	24386
2	["PropertyType"]	5277
3	["Neighbourhood"]	226
4	["Beat"]	87
5	["Zone"]	26
6	["Month"]	25

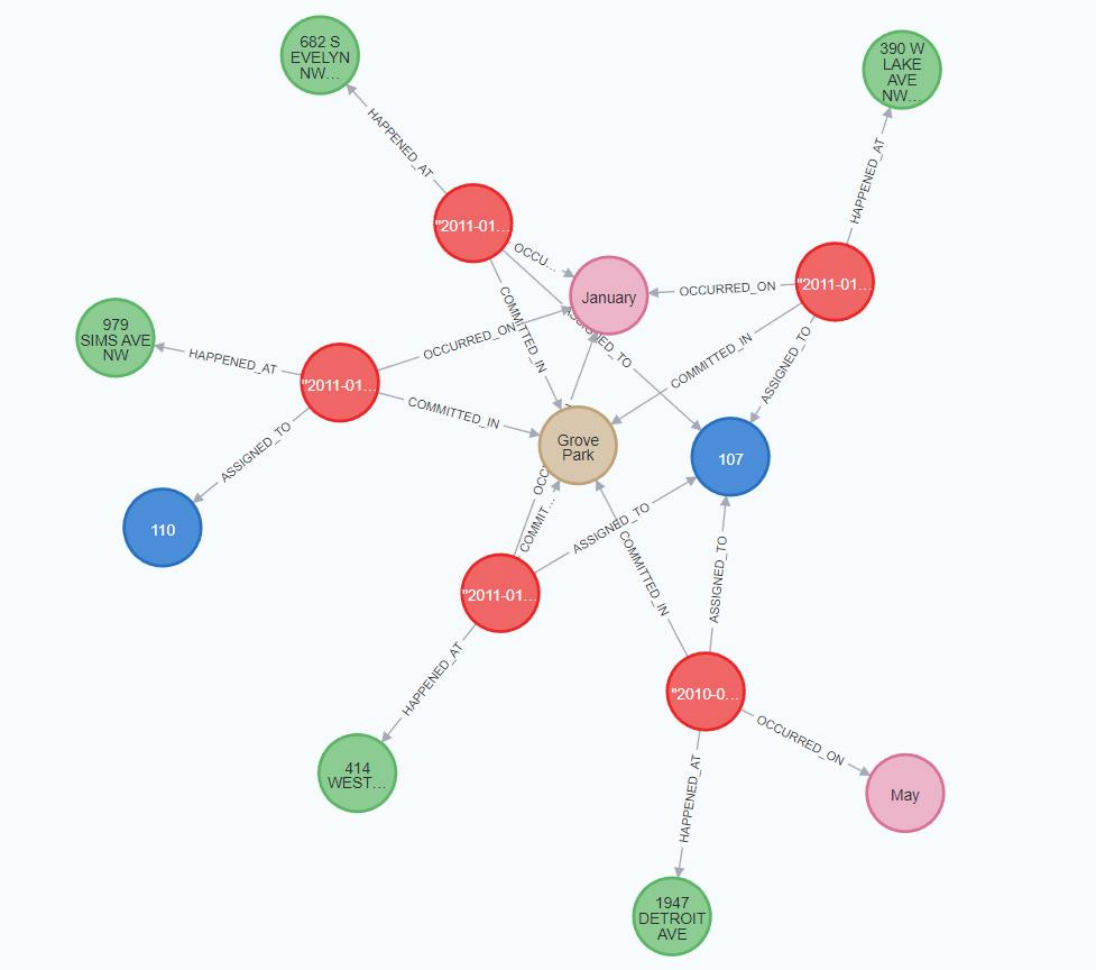
Verification of different colors for each node label



Name: Rohit Kumar Malik
Student ID: 35363998

Graph Structure Visualisation

```
MATCH (c:Crime)
MATCH (c)-[:COMMITTED_IN]->(n:Neighbourhood)
MATCH (c)-[:HAPPENED_AT]->(p:PropertyType)
MATCH (c)-[:OCCURRED_ON]->(m:Month)
MATCH (c)-[:ASSIGNED_TO]->(b:Beat)
RETURN c, n, p, m, b
LIMIT 5;
```



Name: Rohit Kumar Malik
Student ID: 35363998

Provided Cypher Queries & Visualisations

Count Crimes by Type, Neighbourhood, and Month

Query:

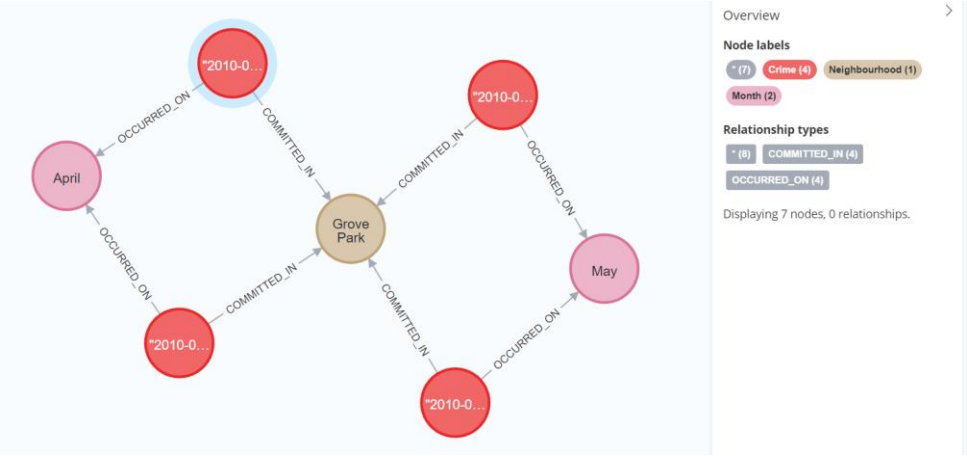
```
MATCH (c:Crime)-[:COMMITTED_IN]->(n:Neighbourhood)
MATCH (c)-[:OCCURRED_ON]->(m:Month)
WHERE c.crime_type = 'house_number'
AND n.name = 'Grove Park'
AND m.year = 2010
AND m.month_name IN ['April', 'May']
RETURN n.name AS Neighbourhood,
       c.crime_type AS Crime_Type,
       count(c) AS Crime_Count,
       collect(m.month_name) AS Months;
```

neo4j$ MATCH (c:Crime)-[:COMMITTED_IN]→(n:Neighbourhood) MATCH (c)-[:OCCURRED_ON]→(m:Month) WHERE c.crime_type = 'house_number' AND n…

Neighbourhood	Crime_Type	Crime_Count	Months
"Grove Park"	"house_number"	4	["May", "May", "April", "April"]

Visualization Query:

```
MATCH (c:Crime)-[:COMMITTED_IN]->(n:Neighbourhood)
MATCH (c)-[:OCCURRED_ON]->(m:Month)
WHERE c.crime_type = 'house_number'
AND n.name = 'Grove Park'
AND m.year = 2010
AND m.month_name IN ['April', 'May']
RETURN c, n, m;
```



Neighbourhoods Sharing Same Crime Types

Query:

```
MATCH (n1:Neighbourhood)<-[:COMMITTED_IN]-(c1:Crime)
MATCH (n2:Neighbourhood)<-[:COMMITTED_IN]-(c2:Crime)
WHERE n1.name < n2.name // Avoid duplicate pairs
AND c1.crime_type = c2.crime_type
WITH n1.name AS Neighbourhood1,
     n2.name AS Neighbourhood2,
     collect(DISTINCT c1.crime_type) AS Common_Crimes
RETURN Neighbourhood1, Neighbourhood2, Common_Crimes,
       size(Common_Crimes) AS Number_Of_Common_Crimes
ORDER BY Number_Of_Common_Crimes DESC
LIMIT 10
```

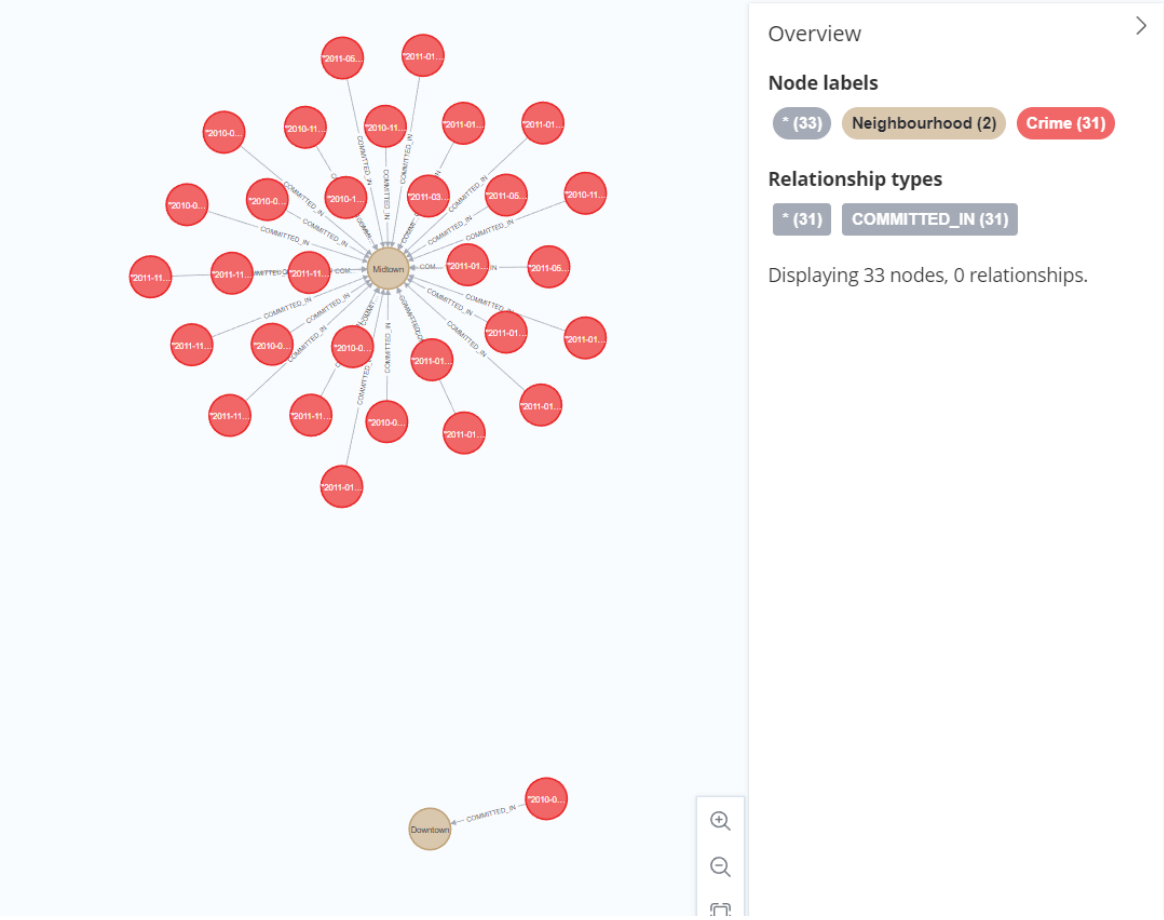
Name: Rohit Kumar Malik
Student ID: 35363998

```
neo4j$ MATCH (n1:Neighbourhood)<-[:COMMITTED_IN]-(c1:Crime) MATCH (n2:Neighbourhood)<-[:COMMITTED_IN]-(c2:Crime) WHERE n1.name < n2.nam…
```

Neighbourhood1	Neighbourhood2	Common_Crimes	Number_Of_Common_Crimes
"Downtown"	"Midtown"	["building", "house_number", "amenity", "road", "office", "tourism", "shop", "leisure"]	8
"Grant Park"	"Lenox"	["shop", "amenity", "road", "tourism", "building", "house_number", "highway"]	7
"Atlantic Station"	"Downtown"	["amenity", "house_number", "building", "tourism", "road", "office", "shop"]	7
"Atlantic Station"	"Midtown"	["building", "house_number", "amenity", "road", "office", "tourism", "shop"]	7
"Garden Hills"	"Lenox"	["shop", "amenity", "road", "tourism", "building", "house_number"]	6
"Inman Park"	"Lenox"	["shop", "amenity", "road", "building", "house_number", "highway"]	6
"Atlantic Station"	"West End"	["house_number", "road", "shop", "amenity", "office", "tourism"]	6
"Midtown"	"West End"	["house_number", "road", "shop", "amenity", "office", "tourism"]	6
"Grant Park"	"Inman Park"	["house_number", "amenity", "building", "highway", "shop", "road"]	6
"Downtown"	"West End"	["house_number", "road", "shop", "amenity", "office", "tourism"]	6

Visualization Query:

```
MATCH (n1:Neighbourhood)<-[:COMMITTED_IN]-(c1:Crime)
MATCH (n2:Neighbourhood)<-[:COMMITTED_IN]-(c2:Crime)
WHERE n1.name < n2.name
AND c1.crime_type = c2.crime_type
WITH n1, n2, collect(DISTINCT c1.crime_type) AS common_crimes
ORDER BY size(common_crimes) DESC
LIMIT 5
MATCH (n1)<-[:COMMITTED_IN]-(c1:Crime)
MATCH (n2)<-[:COMMITTED_IN]-(c2:Crime)
WHERE c1.crime_type = c2.crime_type
AND c1.crime_type IN common_crimes
RETURN n1, n2, c1, c2
LIMIT 30
```



Name: Rohit Kumar Malik
Student ID: 35363998

Top 5 Neighbourhoods for a Specific Crime

Query:

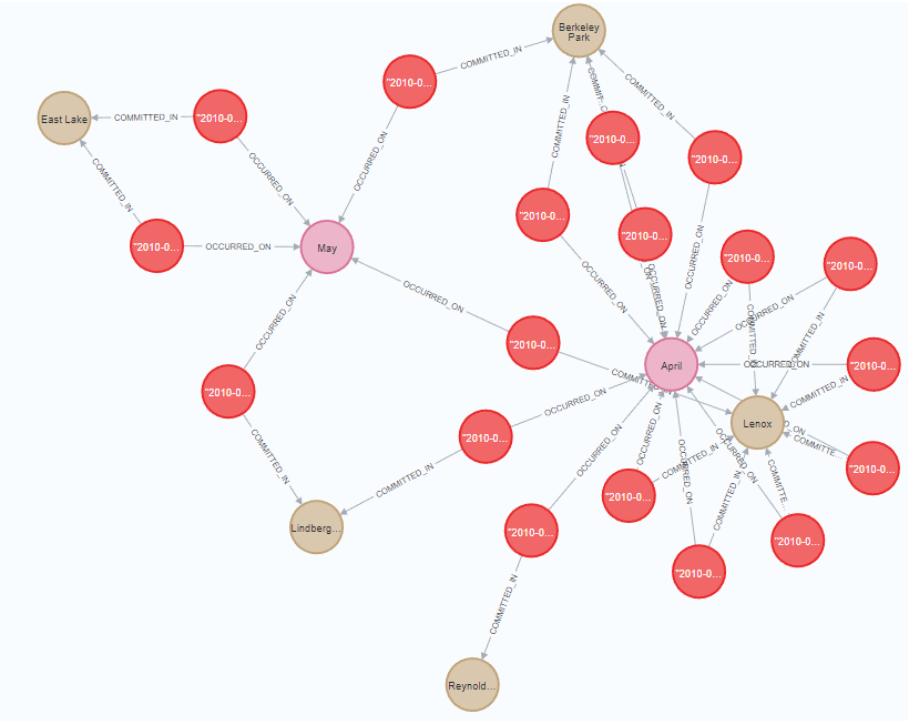
```
// Query 3: Top 5 neighbourhoods for 'shop' in April-May 2010

MATCH (c:Crime)-[:COMMITTED_IN]->(n:Neighbourhood)
MATCH (c)-[:OCCURRED_ON]->(m:Month)
WHERE c.crime_type = 'shop'
AND m.year = 2010
AND m.month_name IN ['April', 'May']
WITH n.name AS Neighbourhood, count(c) AS Crime_Count
ORDER BY Crime_Count DESC
LIMIT 5
RETURN Neighbourhood, Crime_Count
```

neo4j$ // Query 3: Top 5 neighbourhoods for 'shop' in April-May 2010 MATCH (c:Crime)-[:COMMITTED_IN]→(n:Neighbourhood) MATCH (c)-[:OCC…

	Neighbourhood	Crime_Count
1	"Lenox"	8
2	"Berkeley Park"	5
3	"East Lake"	2
4	"Lindbergh/Morosgo"	2
5	"Reynoldstown"	1

Visualization Query:

```
MATCH (c:Crime)-[:COMMITTED_IN]->(n:Neighbourhood)
MATCH (c)-[:OCCURRED_ON]->(m:Month)
WHERE c.crime_type = 'shop'
AND m.year = 2010
AND m.month_name IN ['April', 'May']
WITH n, count(c) AS crime_count
ORDER BY crime_count DESC
LIMIT 5
MATCH (n)<-[:COMMITTED_IN]-(c:Crime)-[:OCCURRED_ON]->(m)
WHERE c.crime_type = 'shop'
AND m.year = 2010
AND m.month_name IN ['April', 'May']
RETURN n, c, m
LIMIT 50
```



Name: Rohit Kumar Malik
Student ID: 35363998

Crime Types per Property Type

Query:

```
MATCH (p:PropertyType)<-[:HAPPENED_AT]-(c:Crime)
WITH p.type AS Property_Address, collect(DISTINCT c.crime_type) AS Crime_Types
RETURN Property_Address, Crime_Types, size(Crime_Types) AS Crime_Type_Count
ORDER BY Crime_Type_Count DESC
LIMIT 10
```

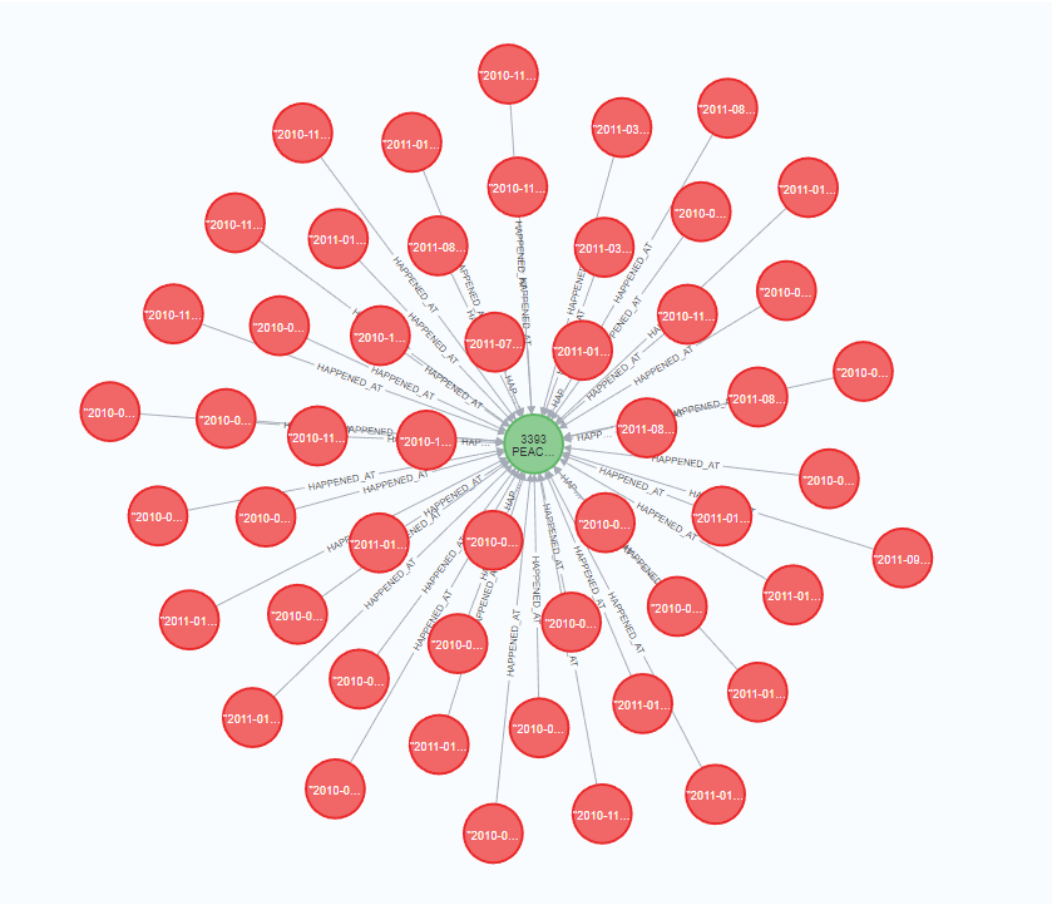
neo4j$ MATCH (p:PropertyType)←[:HAPPENED_AT]-(c:Crime) WITH p.type AS Property_Address, collect(DISTINCT c.crime_type) AS Crime_Types …

Property_Address	Crime_Types	Crime_Type_Count
"3390 FAIRBURN RD SW"	["city", "road", "house_number"]	3
"2685 METROPOLITAN PKWY SW"	["shop", "amenity", "house_number"]	3
"1991 DELOWE DR SW"	["house_number", "road", "city"]	3
"3540 N CAMP CREEK PKWY SW"	["house_number", "road", "city"]	3
"1544 PIEDMONT AVE NE"	["leisure", "amenity", "house_number"]	3
"2950 SPRINGDALE RD SW"	["house_number", "city"]	2
"2420 PIEDMONT RD NE"	["neighbourhood", "amenity"]	2
"3393 PEACHTREE RD NE"	["shop", "road"]	2
"3730 CARMIA DR SW"	["amenity", "shop"]	2
"931 MONROE DR NE"	["building", "amenity"]	2

Visualization Query:

```
MATCH (p:PropertyType)<-[:HAPPENED_AT]-(c:Crime)
WHERE p.type IN [
  '3393 PEACHTREE RD NE',
  '3500 MT GILEAD RD',
  '1024 LITTLE HAMPTON CT SE'
]
RETURN p, c
LIMIT 50
```

Name: Rohit Kumar Malik
Student ID: 35363998



Note: As the dataset does not contains general property categories instead it has specific addresses, I adjusted the query to show crime types at different locations rather than by property type.

Highest Crime Month per Beat

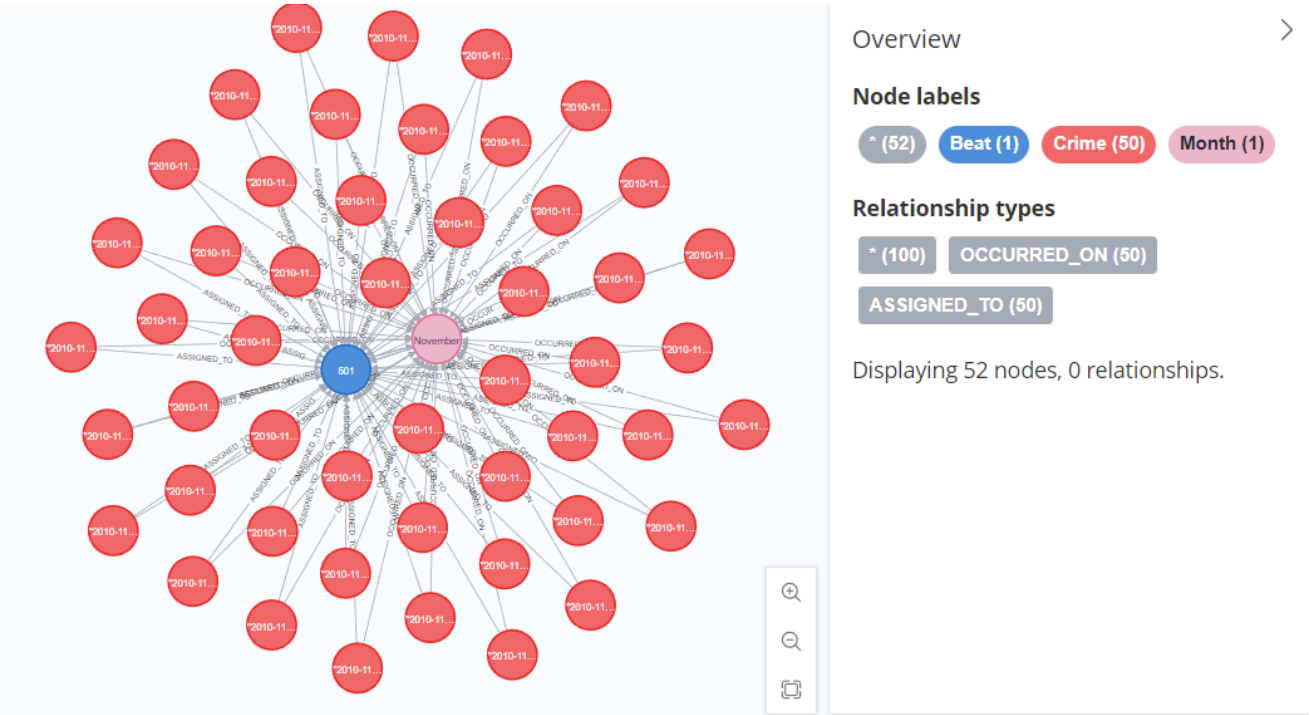
Query:

```
MATCH (b:Beat)<-[:ASSIGNED_TO]-(c:Crime)-[:OCCURRED_ON]->(m:Month)
WHERE m.year = 2010
WITH b.beat_id AS Beat, m.month_name AS Month, count(c) AS Crime_Count
WITH Beat, collect({month: Month, count: Crime_Count}) AS Monthly_Counts
WITH Beat, Monthly_Counts,
     reduce(highest = {month: null, count: 0}, x IN Monthly_Counts |
            CASE WHEN x.count > highest.count THEN x ELSE highest END) AS Highest_Month
RETURN Beat, Highest_Month.month AS Month_With_Highest_Crime, Highest_Month.count AS Crime_Count
ORDER BY Crime_Count DESC
```

Visualization Query:

```
MATCH (b:Beat)<-[:ASSIGNED_TO]-(c:Crime)-[:OCCURRED_ON]->(m:Month)
WHERE m.year = 2010
WITH b, m, count(c) AS crime_count
ORDER BY crime_count DESC
LIMIT 5
MATCH (b)<-[:ASSIGNED_TO]-(c:Crime)-[:OCCURRED_ON]->(m)
WHERE m.year = 2010
RETURN b, c, m
LIMIT 50
```

Name: Rohit Kumar Malik
Student ID: 35363998



Zones with Shared High-Crime Months

Query:

```
MATCH (z1:Zone)<-[:LOCATED_IN]-(n1:Neighbourhood)<-[:COMMITTED_IN]-(c1:Crime)-[:OCCURRED_ON]->(m1:Month)
MATCH (z2:Zone)<-[:LOCATED_IN]-(n2:Neighbourhood)<-[:COMMITTED_IN]-(c2:Crime)-[:OCCURRED_ON]->(m2:Month)
WHERE z1 <> z2 AND m1 = m2
AND m1.year = 2010
WITH z1, z2, m1.month_name AS Month, count(DISTINCT c1) + count(DISTINCT c2) AS Total_Crimes
WITH z1.name AS Zone1, z2.name AS Zone2, Month, Total_Crimes
ORDER BY Total_Crimes DESC
LIMIT 15
RETURN Zone1, Zone2, Month, Total_Crimes
```

neo4j$ MATCH (z1:Zone)←[:LOCATED_IN]-(n1:Neighbourhood)←[:COMMITTED_IN]-(c1:Crime)-[:OCCURRED_ON]→(m1:Month) MATCH (z2:Zone)←[:LOCA…

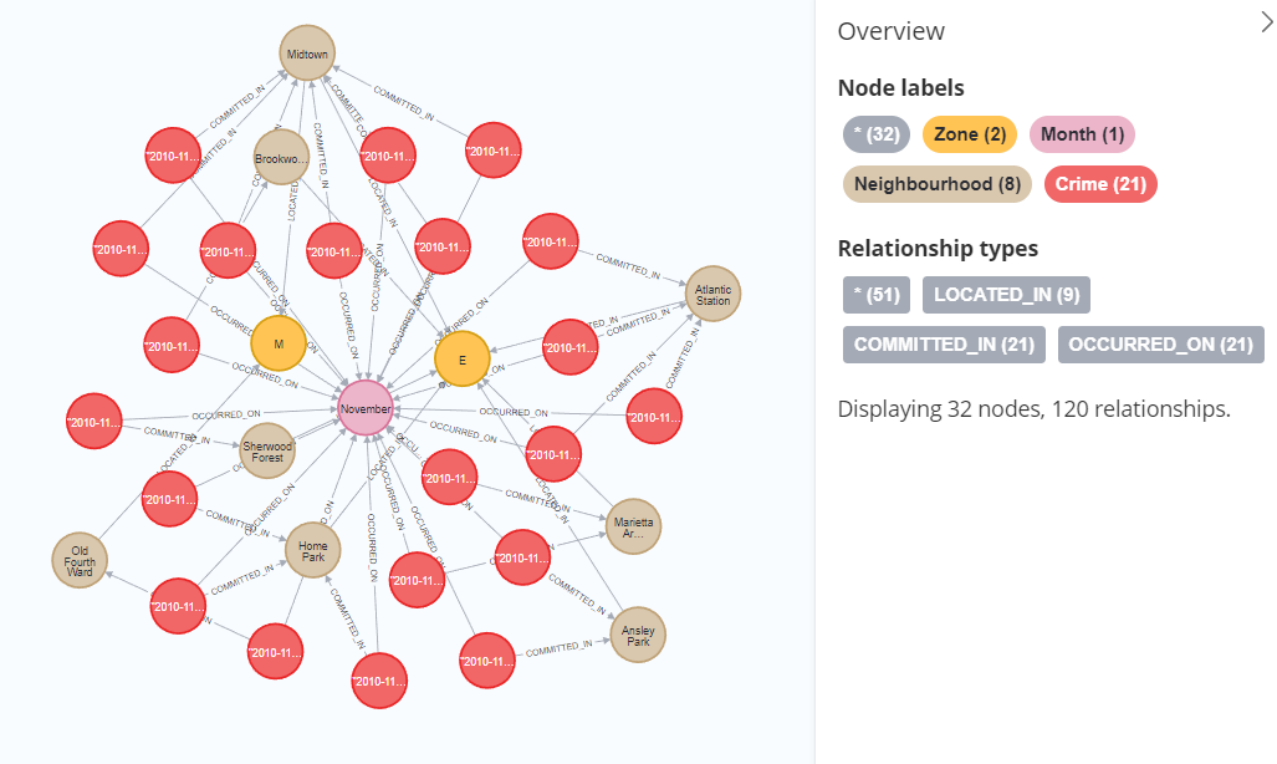
Zone1	Zone2	Month	Total_Crimes
"M"	"E"	"November"	480
"E"	"M"	"November"	480
"B"	"M"	"November"	473
"M"	"B"	"November"	473
"V"	"M"	"November"	428
"M"	"V"	"November"	428
"M"	"R"	"November"	422
"R"	"M"	"November"	422
"T"	"M"	"November"	420
"M"	"T"	"November"	420
"W"	"M"	"November"	415
"M"	"W"	"November"	415
"M"	"J"	"November"	396
"J"	"M"	"November"	396
"X"	"M"	"November"	392

MAX COLUMN WIDTH:

Name: Rohit Kumar Malik
Student ID: 35363998

Visualization Query:

```
MATCH (z1:Zone)<-[:LOCATED_IN]-(n1:Neighbourhood)<-[:COMMITTED_IN]-(c1:Crime)-[:OCCURRED_ON]->(m1:Month)
MATCH (z2:Zone)<-[:LOCATED_IN]-(n2:Neighbourhood)<-[:COMMITTED_IN]-(c2:Crime)-[:OCCURRED_ON]->(m2:Month)
WHERE z1 <> z2 AND m1 = m2
AND m1.year = 2010
WITH z1, z2, m1, count(DISTINCT c1) + count(DISTINCT c2) AS total_crimes
ORDER BY total_crimes DESC
LIMIT 3
MATCH path1 = (z1)<-[:LOCATED_IN]-(n1:Neighbourhood)<-[:COMMITTED_IN]-(c1:Crime)-[:OCCURRED_ON]->(m1)
MATCH path2 = (z2)<-[:LOCATED_IN]-(n2:Neighbourhood)<-[:COMMITTED_IN]-(c2:Crime)-[:OCCURRED_ON]->(m1)
WHERE z1 <> z2
RETURN path1, path2
LIMIT 20
```



Custom Queries with Pathfinding

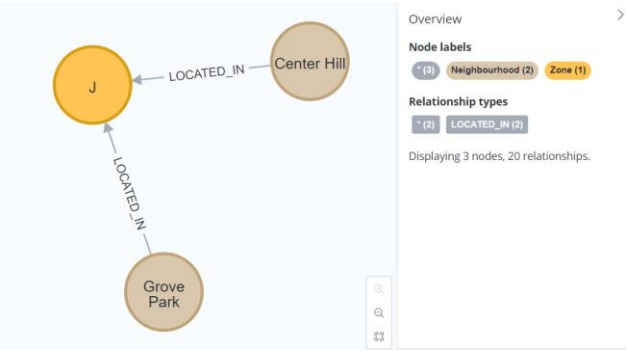
Shortest Path Between High-Crime Neighbourhoods

Query:

```
MATCH (n1:Neighbourhood)<-[:COMMITTED_IN]-(c1:Crime)
WITH n1, count(c1) AS crime_count
WHERE crime_count > 5
WITH n1
LIMIT 5

MATCH (n2:Neighbourhood)<-[:COMMITTED_IN]-(c2:Crime)
WITH n1, n2
WHERE n1 <> n2

MATCH path = shortestPath((n1)-[:LOCATED_IN*1..2]-(n2))
RETURN n1.name AS From_Neighbourhood,
       n2.name AS To_Neighbourhood,
       path,
       length(path) AS Path_Length
ORDER BY Path_Length
LIMIT 10
```

Visualization Query:

Name: Rohit Kumar Malik
Student ID: 35363998



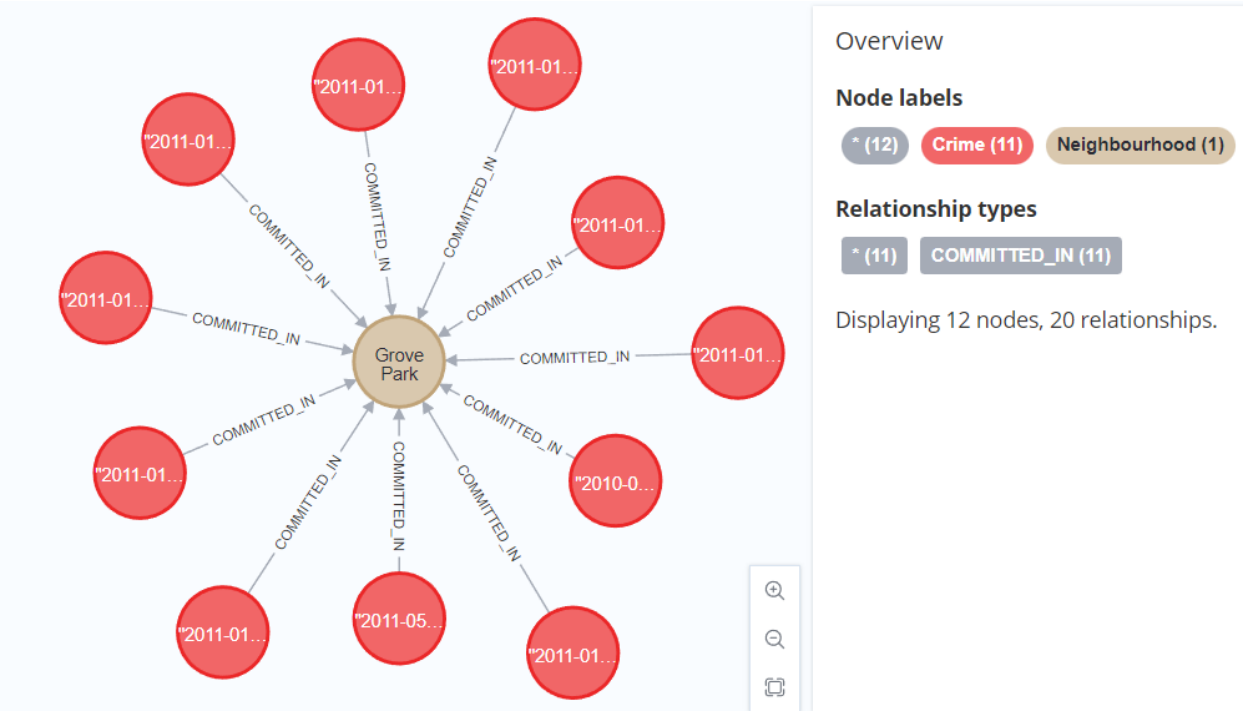
Pattern Between Different Crime Types Through Locations

Query:

```
MATCH (c1:Crime)-[:COMMITTED_IN]->(n:Neighbourhood)<-[:COMMITTED_IN]-(c2:Crime)
WHERE c1.crime_type <> c2.crime_type
  AND c1.crime_id < c2.crime_id

MATCH path = shortestPath((c1)-[:COMMITTED_IN|HAPPENED_AT*1..4]-(c2))
WHERE length(path) >= 2

RETURN c1.crime_type AS Crime_Type1,
       c2.crime_type AS Crime_Type2,
       n.name AS Shared_Neighbourhood,
       length(path) AS Path_Length,
       path
LIMIT 10
```

Visualization Query:

```
MATCH (c1:Crime)-[:COMMITTED_IN]->(n:Neighbourhood)<-[:COMMITTED_IN]-(c2:Crime)
WHERE c1.crime_type <> c2.crime_type
  AND c1.crime_id < c2.crime_id
  AND c1.crime_type IS NOT NULL
  AND c2.crime_type IS NOT NULL

MATCH path = shortestPath((c1)-[:COMMITTED_IN|HAPPENED_AT*1..4]-(c2))
WHERE length(path) >= 2

RETURN path
LIMIT 10
```



Name: Rohit Kumar Malik
Student ID: 35363998

Design Choices – Discussion and Justification

For this assignment, my main aim was to design a graph structure which is easy to understand and powerful enough to answer all the required queries. I started by identifying and analysing the dataset the most meaningful entities. Then, I used the Arrow tool to design the schema visually, keeping the nodes like Crime, Neighbourhood, Zone, PropertyType, Beat, and Month clearly separated. Each of these node types had its own CSV generated using my Python preprocessing script.

In the Python code, I renamed the original column names into clearer name, and in more consistent ones. I also included steps to convert the date column into proper month, year, and month_name, which helped in forming the Month node for answering time-based queries.

I didn't treat every field as a separate node. I made only the ones that made sense for relationships and analysis. For example, property location strings, I did not split into coordinates or categories, since they were just addresses. Instead, they were used in the PropertyType node.

When creating the database in Neo4j, I verified that each node label had a different colour. While Neo4j automatically assigns some of these, I verified their visual distinction in the Browser. I also added constraints for uniqueness avoid duplicates and to improve performance.

In terms of limitations, the dataset has some specific property addresses rather than generic types like "commercial" or "residential", which made it difficult to group crimes meaningfully by property type.

Overall, the design is structured to balance query support and clarity, and which helped me to answer all six provided queries and two meaningful custom ones using the graph.